

The Transformer

A Visual Journey from RNNs to Deep Encoders & Decoders Story

By : Kallind Soni & Yuvrajsinh Bodana

(PhD CSE 1st Year, MS By Research CSE 2nd Year)

LTRC, IIIT Hyderabad

If we have a GPU with 5,000 cores, how many steps does an RNN take to process a 100-word sentence?

- A) 1 step (fully parallelized)
- B) 5,000 steps
- C) 100 steps (one sequential step per word)
- D) It depends on the memory

If we have a GPU with 5,000 cores, how many steps does an RNN take to process a 100-word sentence?

A) 1 step (fully parallelized)

B) 5,000 steps

C) 100 steps (one sequential step per word)

D) It depends on the memory

The Prologue: The Limits of RNNs & LSTMs



Sequential Bottleneck

RNNs and LSTMs process data one step at a time. To understand word 100, they must sequentially process words 1 through 99.



Hard to Parallelize

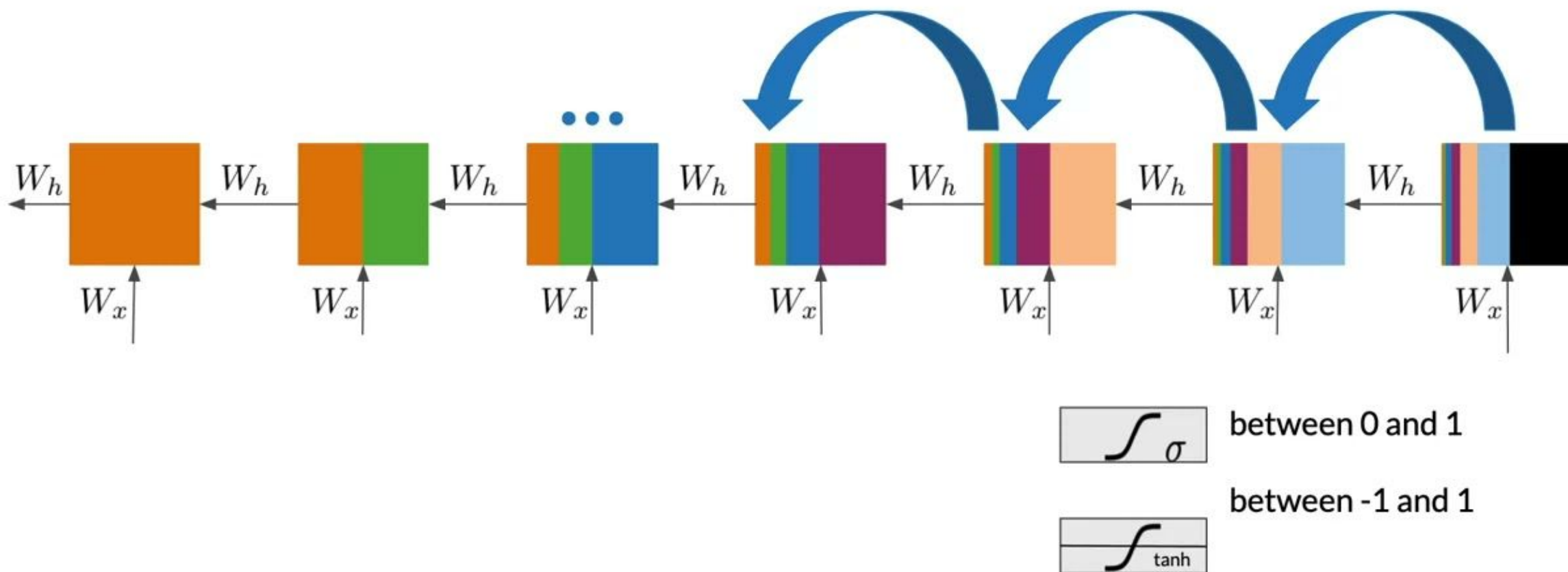
Because of this sequential nature, they cannot take full advantage of modern GPU hardware, drastically slowing down training.



Vanishing Memory

Despite LSTMs attempting to fix this, models still suffer from the "vanishing gradient" problem, struggling to remember context from far back.

Backpropagation through time



Attention Is All You Need

Ashish Vaswani*

Google Brain

avaswani@google.com

Noam Shazeer*

Google Brain

noam@google.com

Niki Parmar*

Google Research

nikip@google.com

Jakob Uszkoreit*

Google Research

usz@google.com

Llion Jones*

Google Research

llion@google.com

Aidan N. Gomez* †

University of Toronto

aidan@cs.toronto.edu

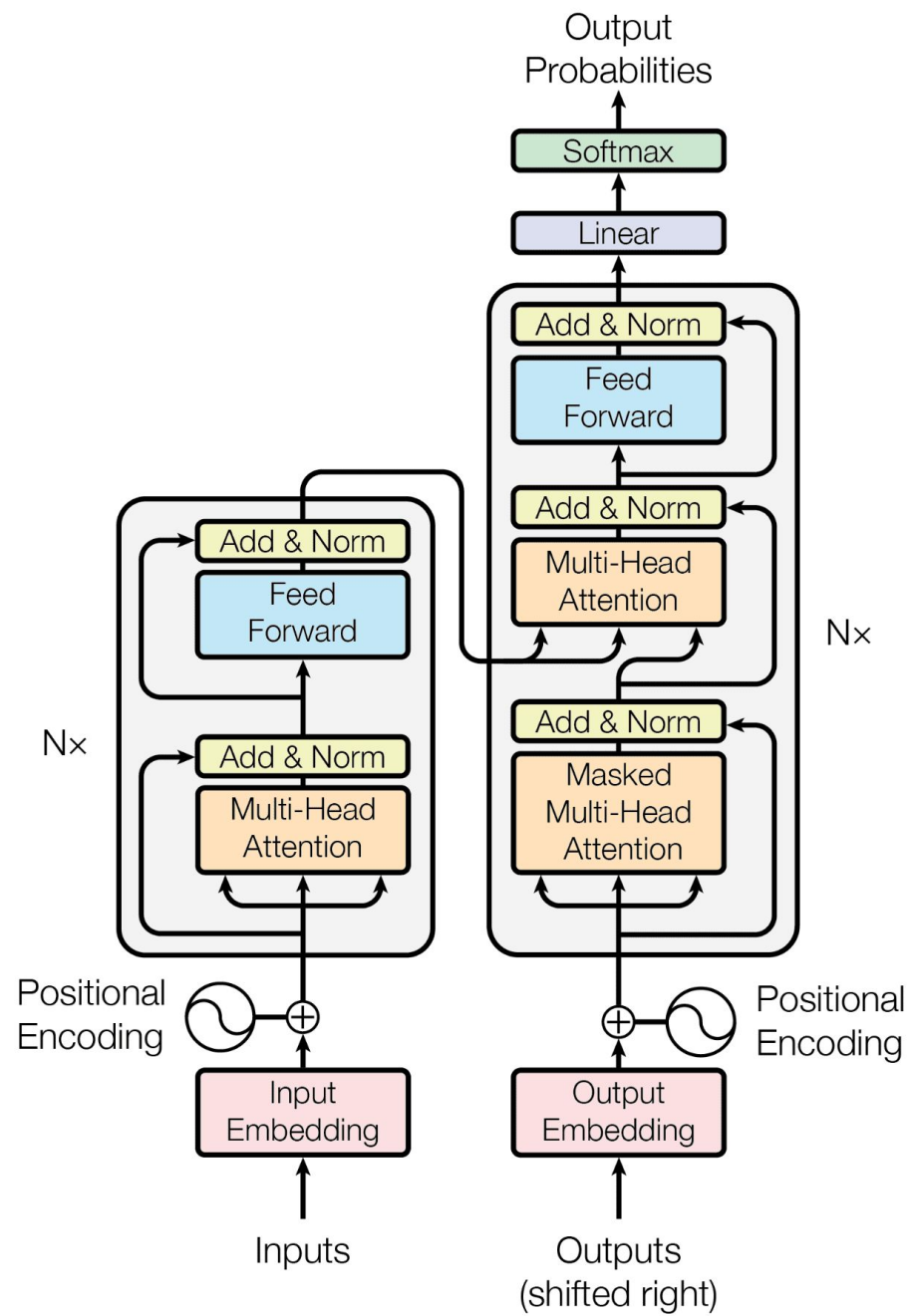
Łukasz Kaiser*

Google Brain

lukaszkaizer@google.com

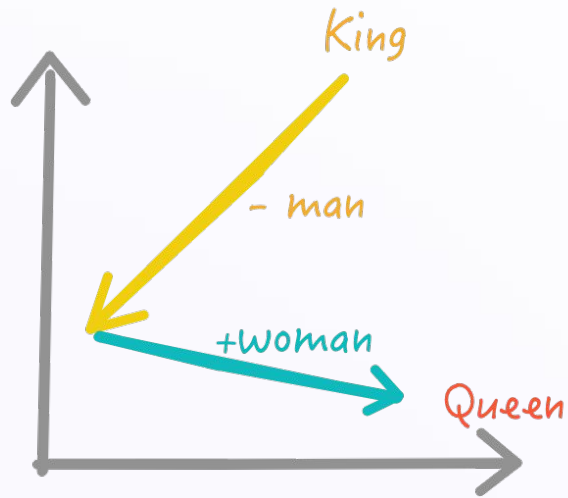
Illia Polosukhin* ‡

illia.polosukhin@gmail.com

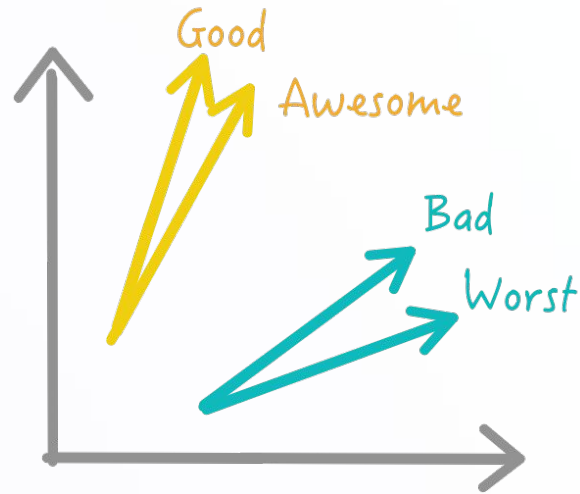


The Language of Machines:

Embeddings



a) Learns Analogy



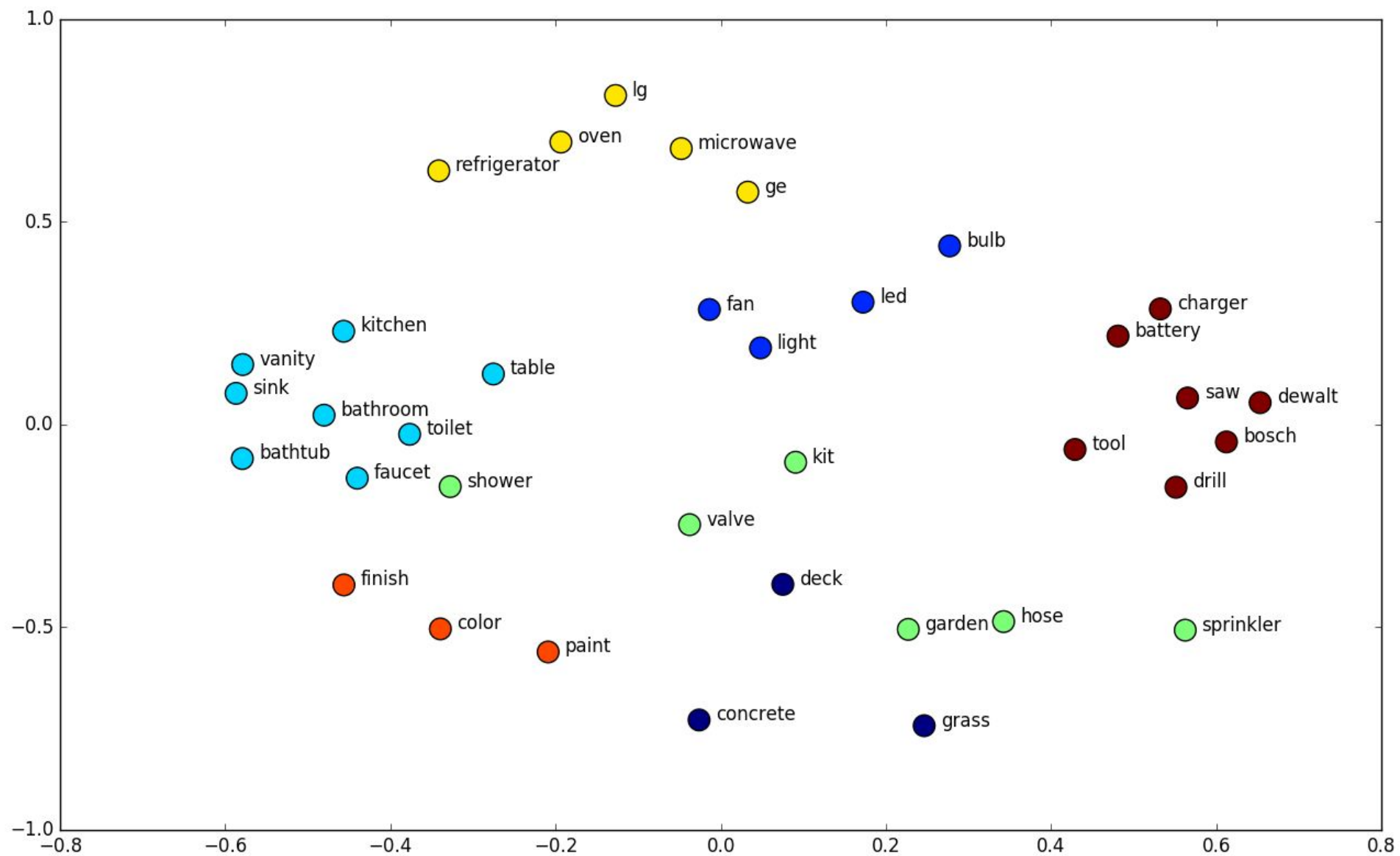
b) Similar Words have same angles

Why map words to space?

Computers do not understand text. They require numbers. But simple IDs (e.g., Apple=1, King=2) don't capture any meaning.

Word Embeddings map discrete words into a continuous, high-dimensional space (e.g., 512 dimensions). In this space, words with similar meanings cluster together.

This allows the architecture to grasp semantic relationships, understanding that "King" is to "Man" what "Queen" is to "Woman".

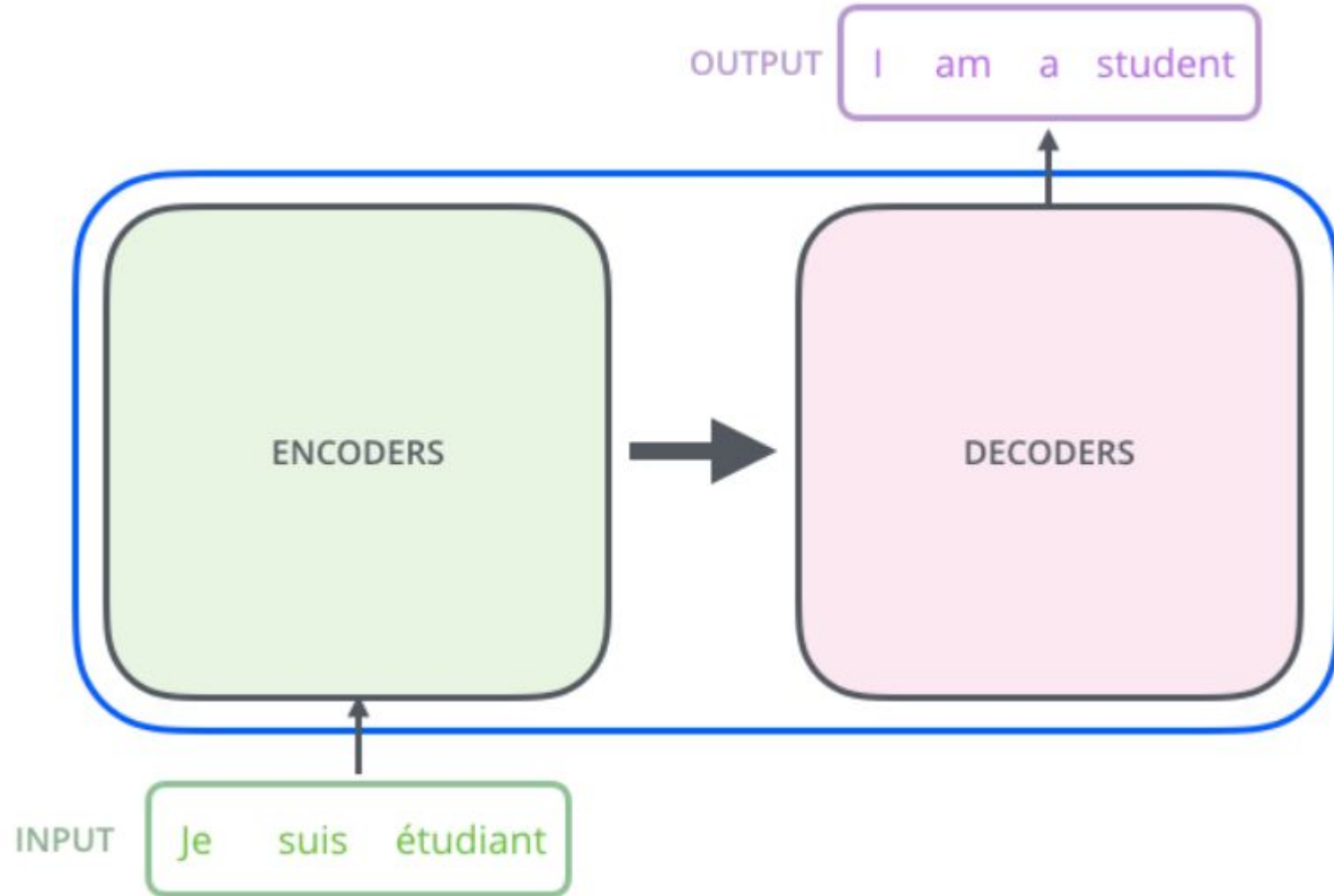


We know that **King - Man + Woman = Queen** in a well-trained embedding space. What do you think you would get if you calculated:

Paris - France + Japan = ?

We know that **King - Man + Woman = Queen** in a well-trained embedding space. What do you think you would get if you calculated:

Paris - France + Japan = Tokyo



The Evolution of Attention

2015

Encoder-Decoder Attention

The decoder is allowed to
"look back" and weigh all
input words while
generating translation.

2022

Decoder-only LLMs

Eliminating the need of a
seperate encoder and
allowing auto regressive
text generation

2014

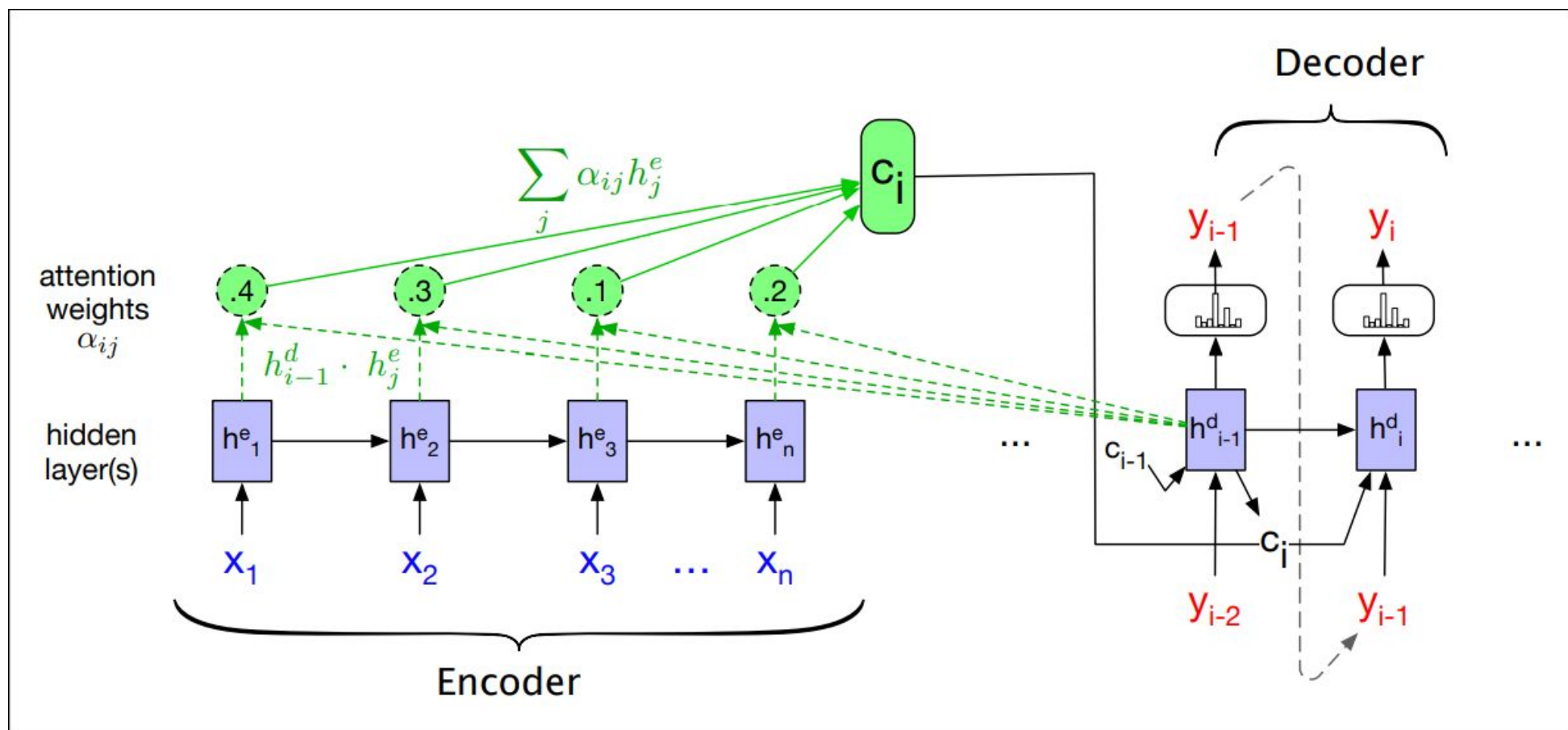
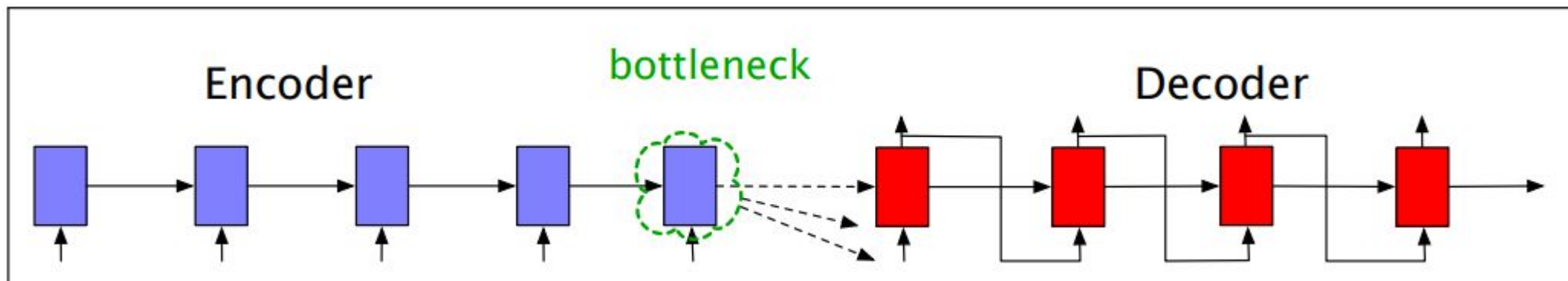
Seq2Seq Model

The entire input sequence
is compressed into a
single, bottlenecked
context vector.

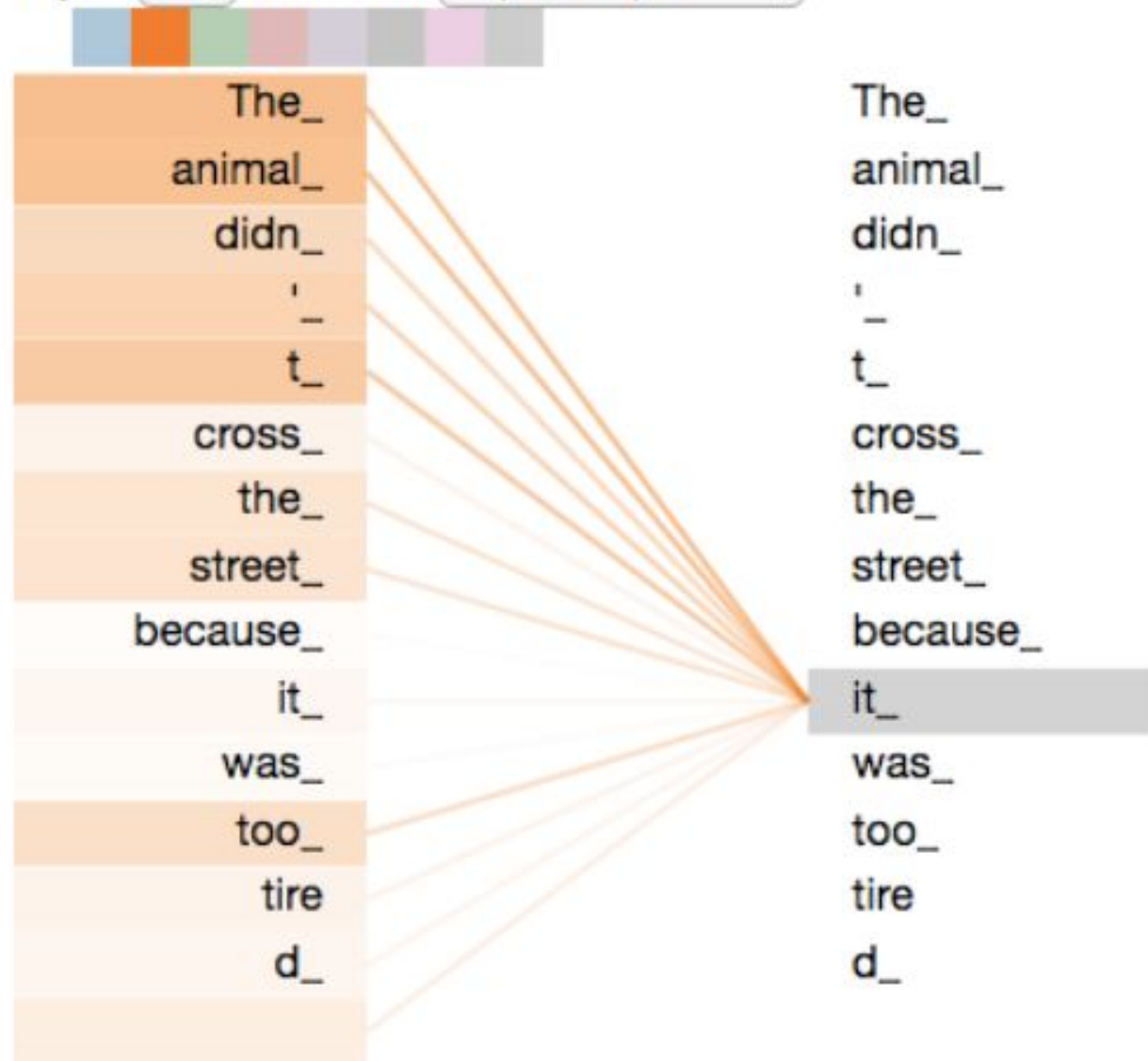
2017

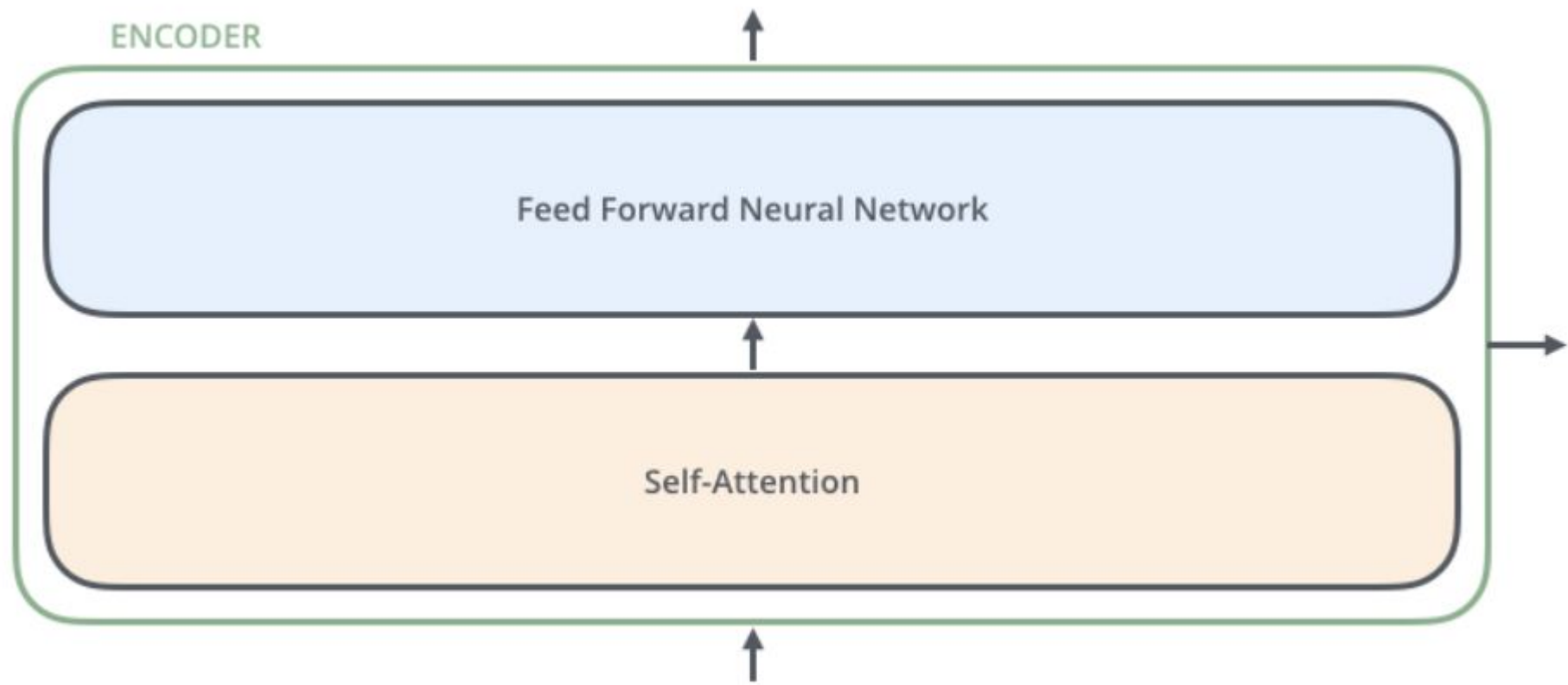
Self-Attention - The Transformer

RNNs are discarded entirely.
"Attention is All You Need"
becomes the new standard.



Layer: 5 Attention: Input - Input





Self-Attention & The QKV Engine

To relate words to each other, self-attention uses a system inspired by database retrieval:



Query (Q)

What the current word is looking for.

"I am an adjective, where is my noun?"



Key (K)

What other words have to offer.

"I am a noun describing an animal."

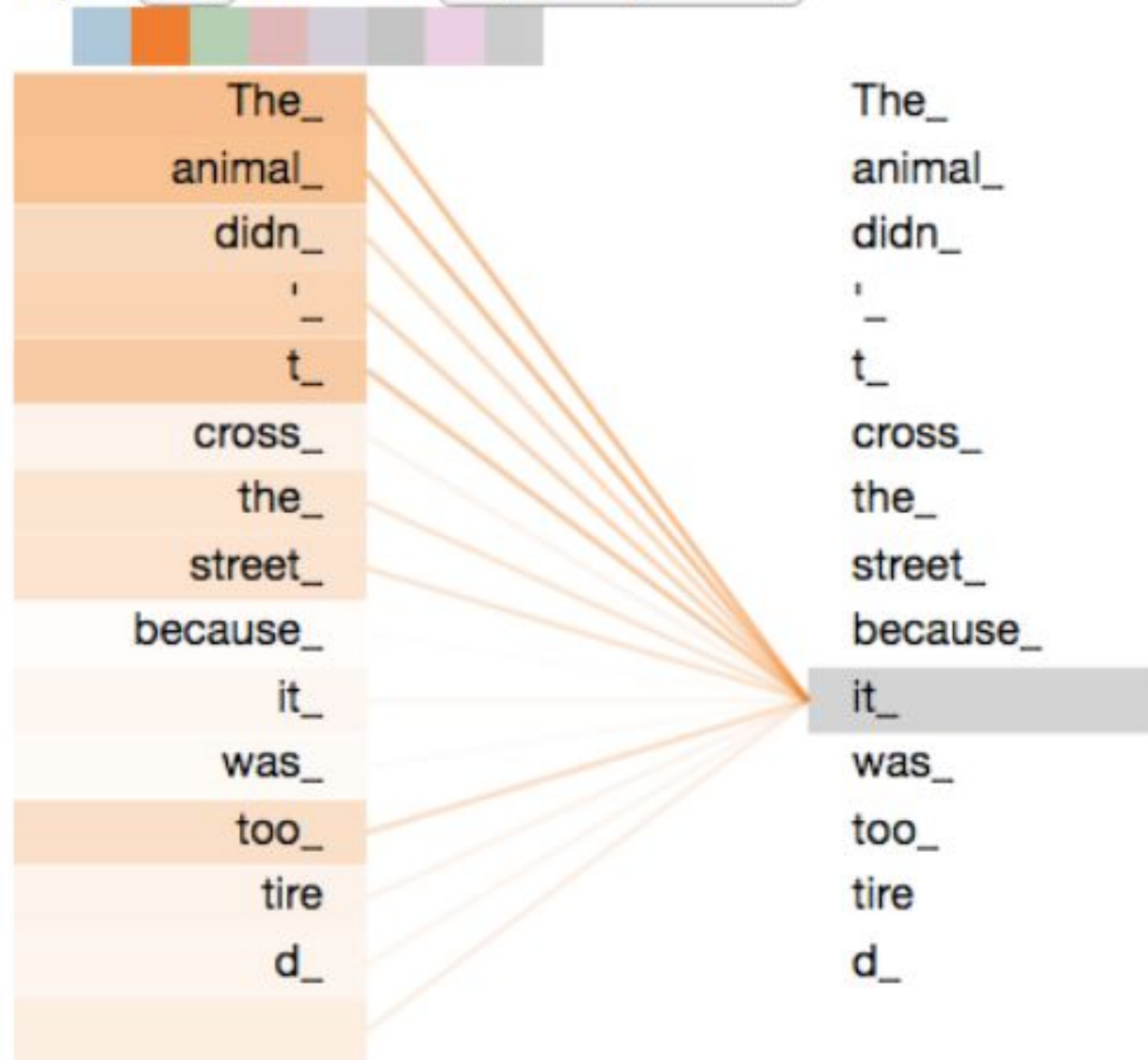


Value (V)

The actual semantic content passed along if Query and Key match.

The "payload" of information.

Layer: 5 Attention: Input - Input

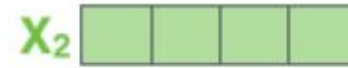
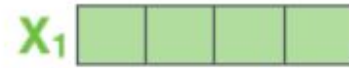


Input

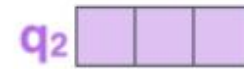
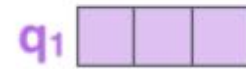
Thinking

Machines

Embedding

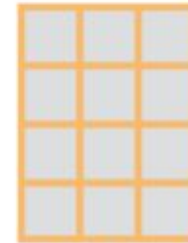
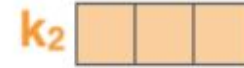
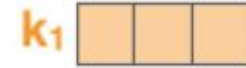


Queries



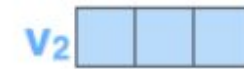
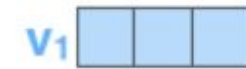
W^Q

Keys



W^K

Values



W^V

Input

Embedding

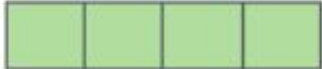
Queries

Keys

Values

Score

Thinking

x_1 

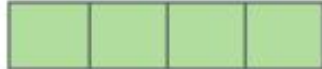
q_1 

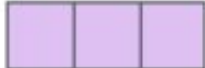
k_1 

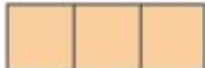
v_1 

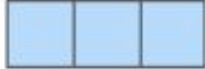
$$q_1 \cdot k_1 = 112$$

Machines

x_2 

q_2 

k_2 

v_2 

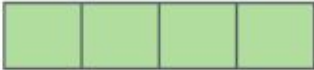
$$q_1 \cdot k_2 = 96$$

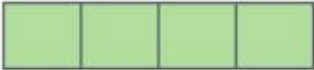
Input

Thinking

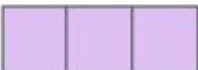
Machines

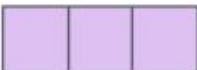
Embedding

x_1 

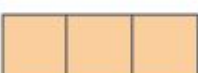
x_2 

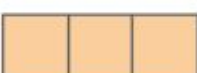
Queries

q_1 

q_2 

Keys

k_1 

k_2 

Values

v_1 

v_2 

Score

$$q_1 \cdot k_1 = 112$$

$$q_1 \cdot k_2 = 96$$

Divide by 8 ($\sqrt{d_k}$)

14

12

Softmax

0.88

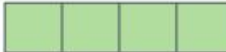
0.12

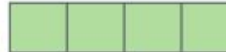
Input

Thinking

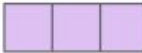
Machines

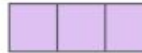
Embedding

x_1 

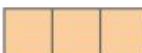
x_2 

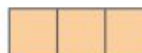
Queries

q_1 

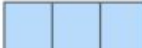
q_2 

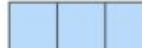
Keys

k_1 

k_2 

Values

v_1 

v_2 

Score

$q_1 \cdot k_1 = 112$

$q_1 \cdot k_2 = 96$

Divide by 8 ($\sqrt{d_k}$)

14

12

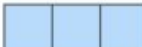
Softmax

0.88

0.12

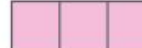
Softmax

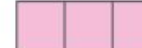
X
Value

v_1 

v_2 

Sum

z_1 

z_2 

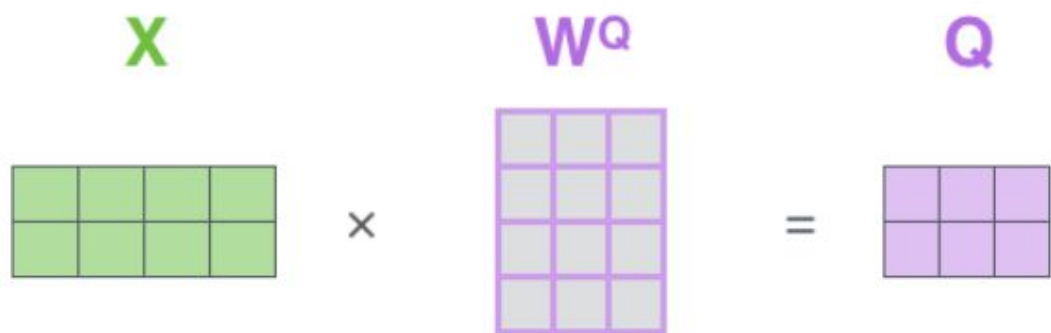
$$X \times W^Q = Q$$


Diagram illustrating the calculation of the Query matrix Q . The input matrix X (green, 2x4) is multiplied by the weight matrix W^Q (purple, 4x4) to produce the Query matrix Q (purple, 2x4).

$$X \times W^K = K$$


Diagram illustrating the calculation of the Key matrix K . The input matrix X (green, 2x4) is multiplied by the weight matrix W^K (orange, 4x4) to produce the Key matrix K (orange, 2x4).

$$X \times W^V = V$$


Diagram illustrating the calculation of the Value matrix V . The input matrix X (green, 2x4) is multiplied by the weight matrix W^V (blue, 4x4) to produce the Value matrix V (blue, 2x4).

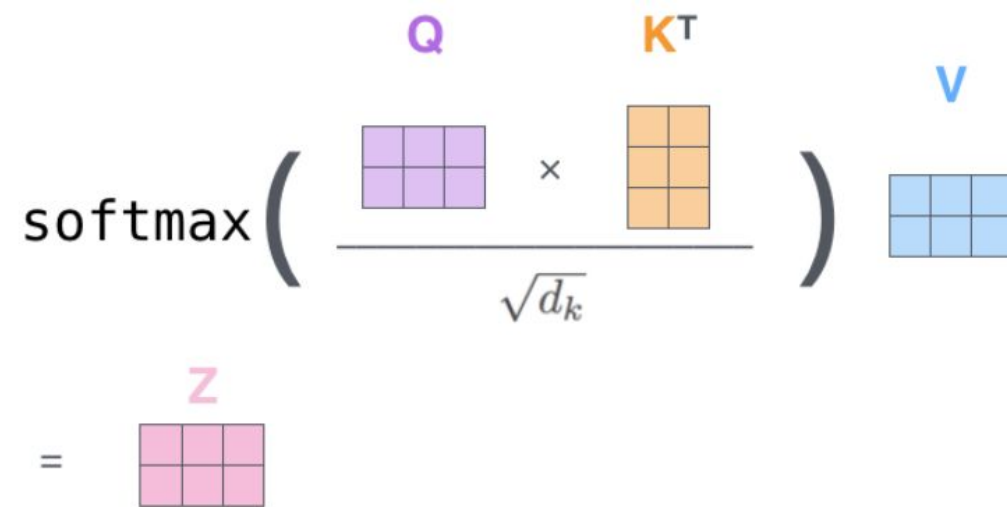
$$\text{softmax}\left(\frac{Q \times K^T}{\sqrt{d_k}}\right) \times V = Z$$


Diagram illustrating the self-attention calculation in matrix form. The Query matrix Q (purple, 2x4) is multiplied by the transposed Key matrix K^T (orange, 4x2). The result is divided by the square root of the dimension d_k ($\sqrt{d_k}$). The softmax function is applied to the result, and then the Value matrix V (blue, 2x4) is multiplied by the output to produce the final self-attention output Z (pink, 2x4).

The self-attention calculation in matrix form

$$\text{Attention} (Q , K , V) = \text{softmax} \frac{QK^T}{\sqrt{d_k}} V$$

Multi-Head Attention:

Perspectives

Instead of performing a single attention function, the model projects Q, K, and V multiple times in parallel ("heads"). This allows the model to jointly attend to information from different representation subspaces.



Head 1: Syntax

One attention head might focus strictly on grammatical structure, linking verbs to their subjects or prepositions to objects.



Head 2: Coreference

Another head might specialize in resolving pronouns, connecting words like "it" or "she" to the correct entity mentioned earlier.



Head 3: Semantics

A third head could look for deeper emotional or semantic relationships, linking adjectives with the overall sentiment.

Look at this sentence: ***'The bank clerk approved the loan because it was fully collateralized.'***

Which attention head (Syntax, Coreference, or Semantics) is responsible for figuring out what the word **'it'** refers to?

- A) Syntax
- B) Coreference
- C) Semantics

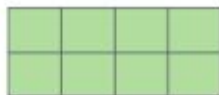
Look at this sentence: ***'The bank clerk approved the loan because it was fully collateralized.'***

Which attention head (Syntax, Coreference, or Semantics) is responsible for figuring out what the word **'it'** refers to?

- A) Syntax
- B) Coreference**
- C) Semantics

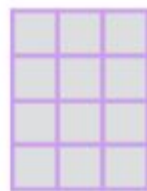
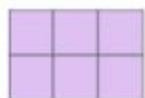
X

Thinking
Machines



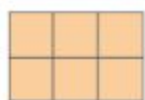
ATTENTION HEAD #0

Q_0



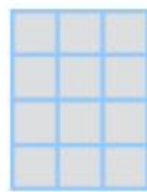
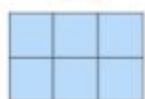
W_0^Q

K_0



W_0^K

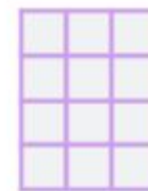
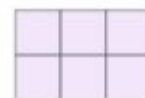
V_0



W_0^V

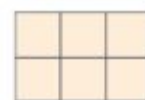
ATTENTION HEAD #1

Q_1



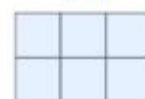
W_1^Q

K_1



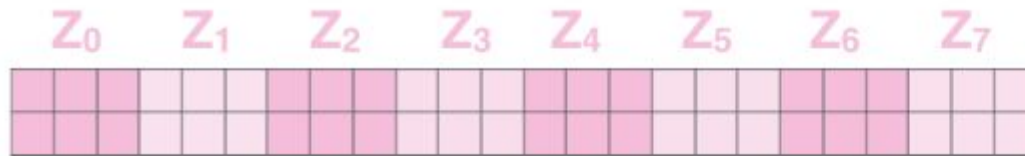
W_1^K

V_1



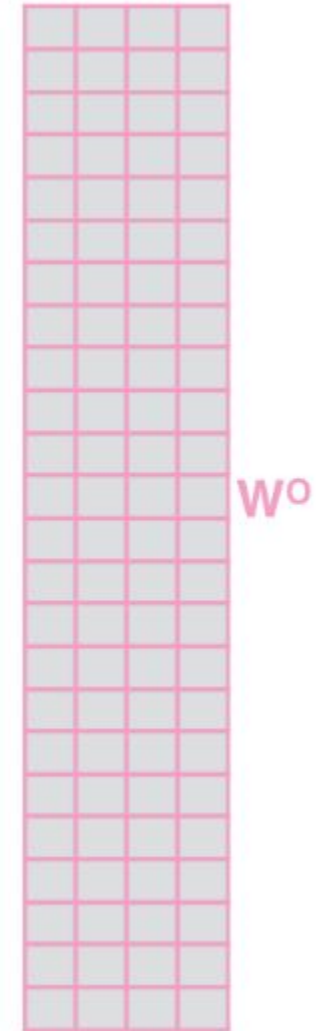
W_1^V

1) Concatenate all the attention heads

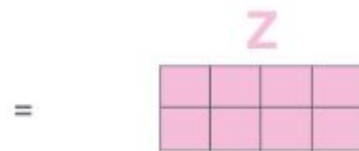


2) Multiply with a weight matrix W^O that was trained jointly with the model

$\times$



3) The result would be the Z matrix that captures information from all the attention heads. We can send this forward to the FFNN



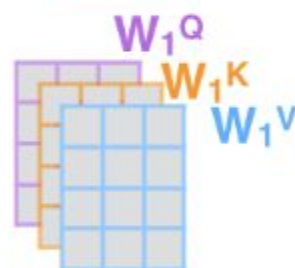
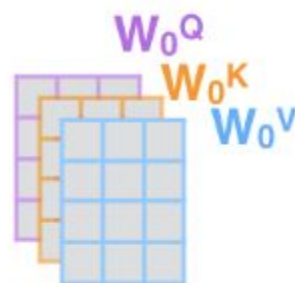
1) This is our input sentence*

Thinking
Machines

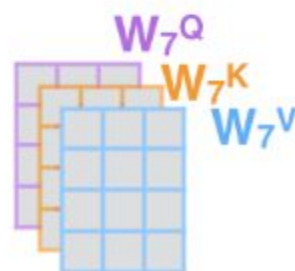
2) We embed each word*



3) Split into 8 heads.
We multiply X or R with weight matrices



...



4) Calculate attention using the resulting $Q/K/V$ matrices



...



5) Concatenate the resulting Z matrices, then multiply with weight matrix W^O to produce the output of the layer



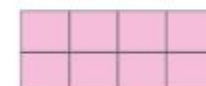
...



W^O



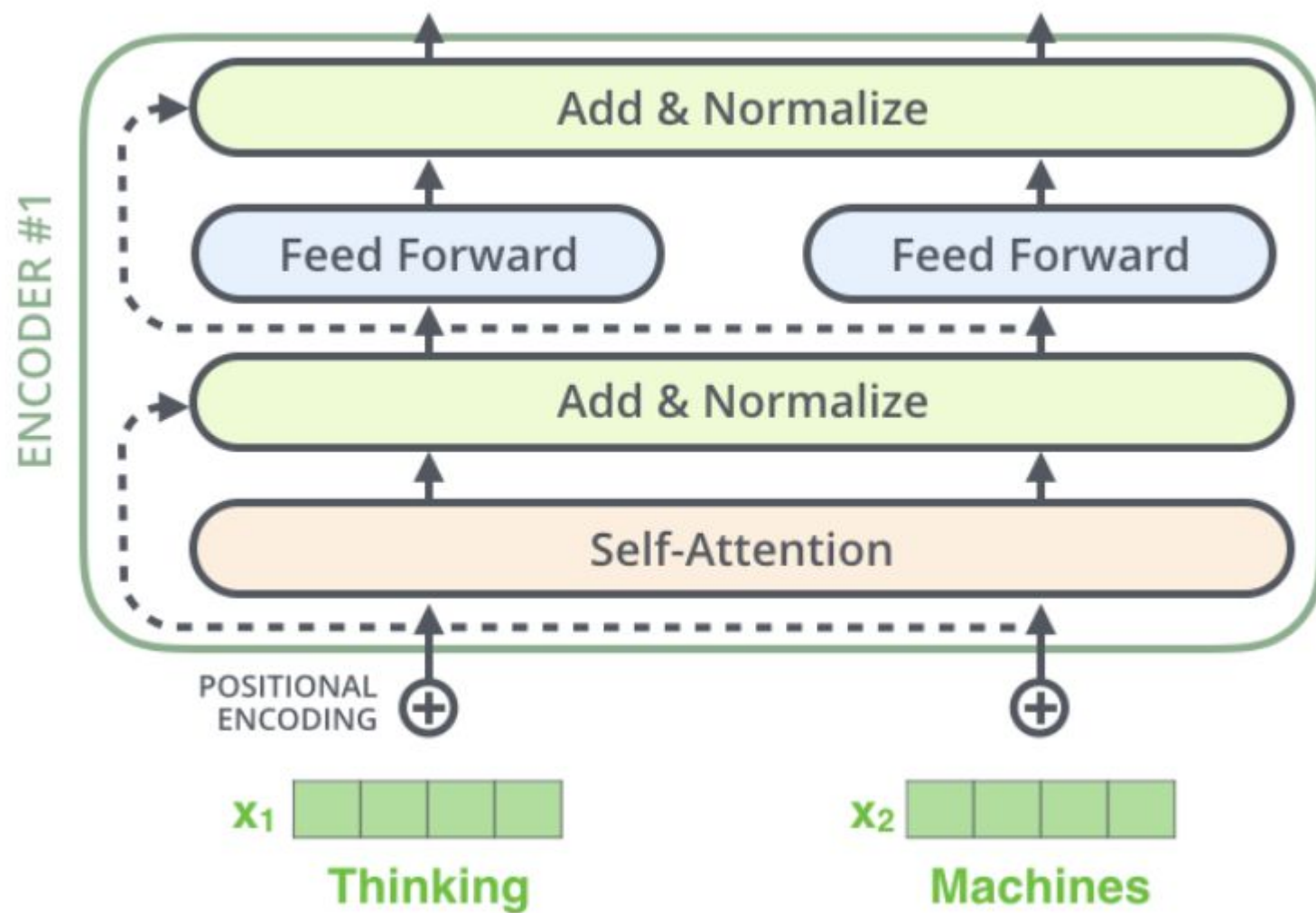
Z



* In all encoders other than #0, we don't need embedding. We start directly with the output of the encoder right below this one



Residuals



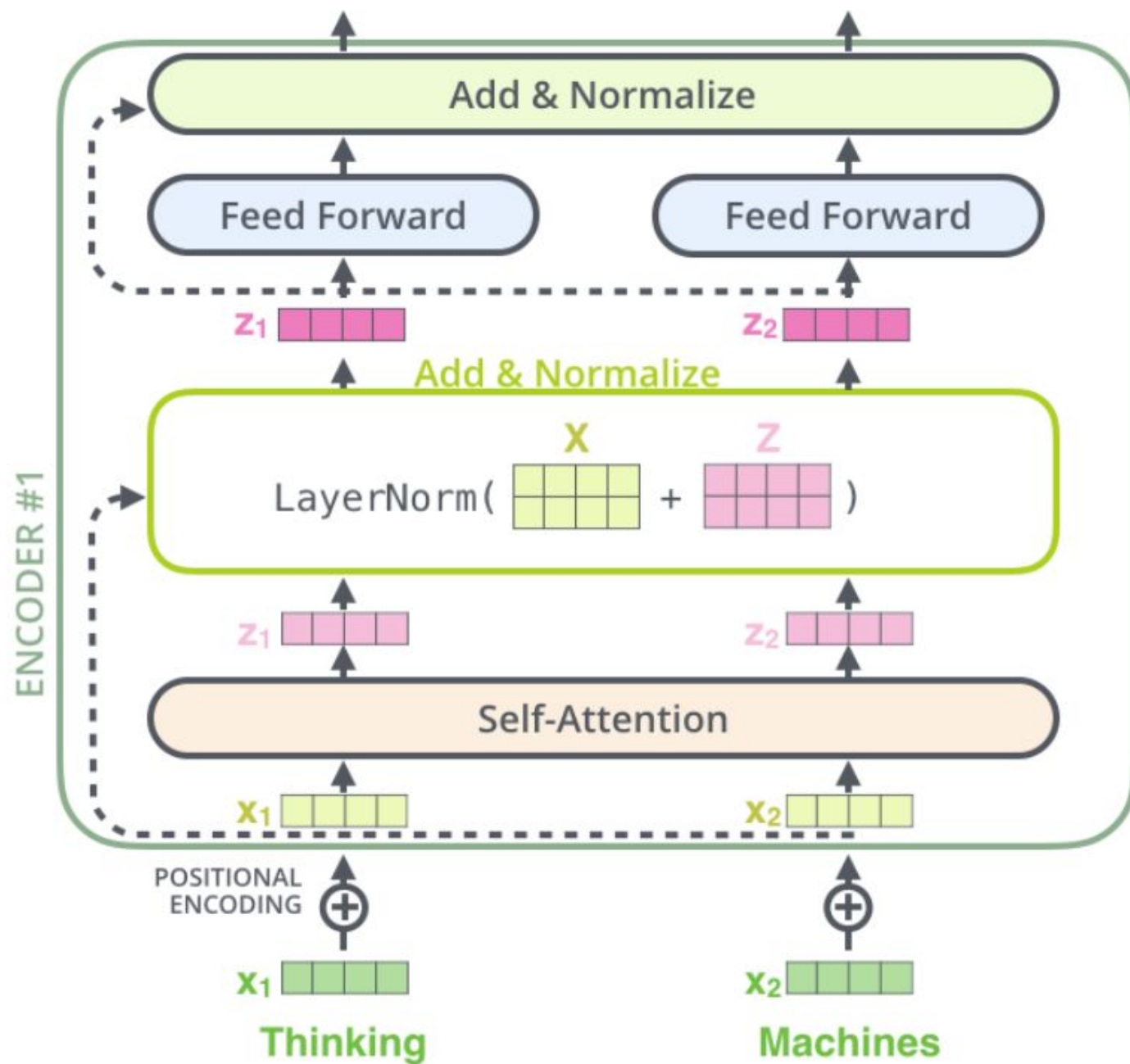
Preserving the Past: Residuals

As neural networks get deeper, the original information (and gradients) can get lost. Residual connections solve this by adding the original input back to the output of the sublayer, bypassing the complex math.

$$y_{\text{out}} = x + \text{Sublayer} (x)$$

Ensures that even in a 96-layer architecture, the model never "forgets" the original word.

How to control variance?



Layer Normalization: Pre vs. Post

Post-Norm (The Original)

In the original 2017 architecture, normalization happens *after* the residual addition.

$$y = \text{LayerNorm} (x + \text{Sublayer} (x))$$

Challenge: Hard to train deep models. The variance grows, requiring careful learning rate warmup schedules.

Pre-Norm (Modern Standard)

In modern models (like GPT-3), normalization happens *before* the sublayer transformation.

$$y = x + \text{Sublayer} (\text{LayerNorm} (x))$$

Advantage: Gradients flow directly through the residual path. Highly stable at initialization; no warmup needed.

Layer Normalization: Math & Parameters

The Normalization Step

Calculating the mean (μ) and variance (σ^2) across the feature dimensions for a single training example.

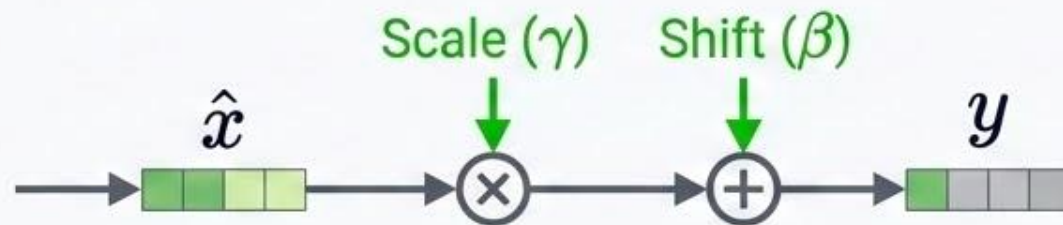
$$\hat{x} = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}}$$

where ϵ is a small constant for stability.

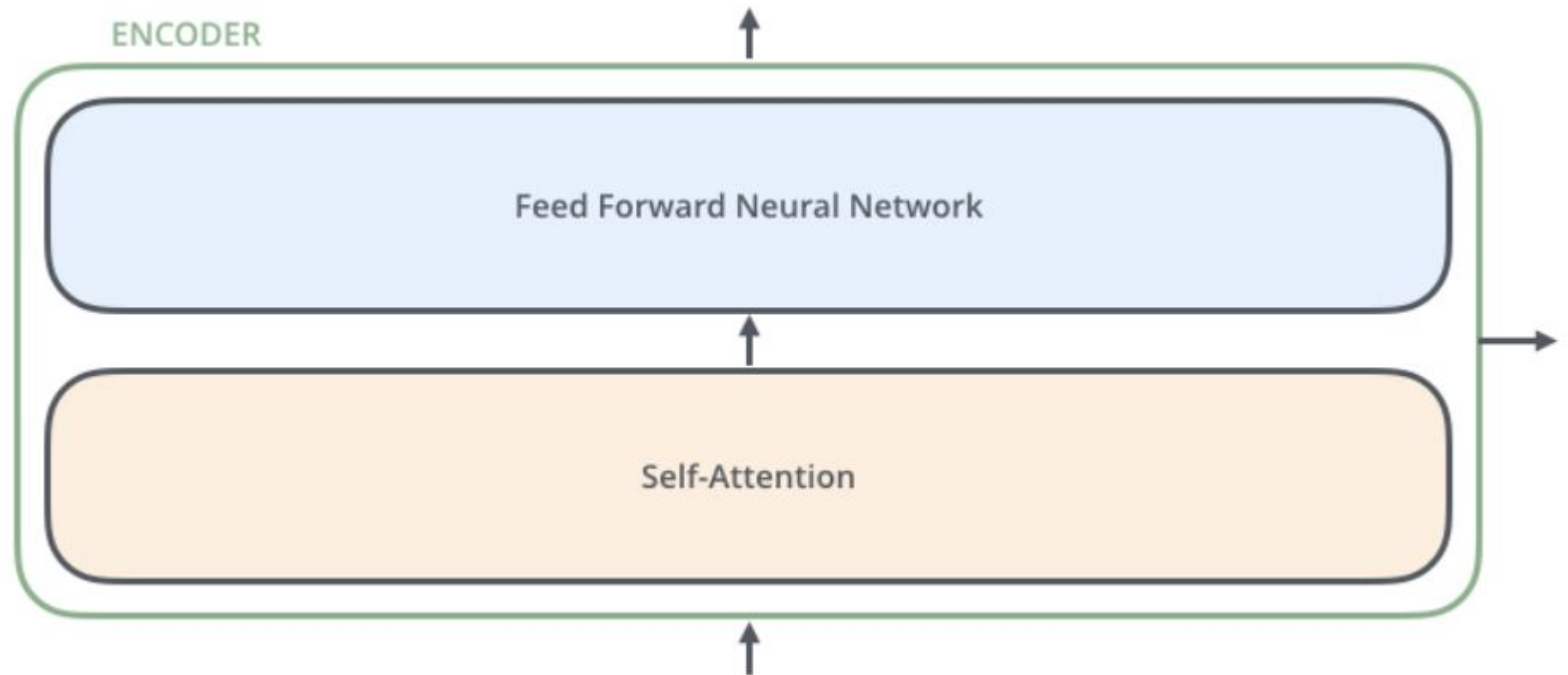
Learnable Scale & Shift

Introducing trainable parameters (γ and β) that allow the model to re-introduce useful variance and shift.

$$y = \gamma \hat{x} + \beta$$



Getting the Vocabulary Dimensions Back!



The Feed-Forward Network (FFN)

2x

Linear Transforms

Thinking Deeper

If self-attention is how words *communicate* with each other, the Feed-Forward layer is how each word individually *thinks* about that new information.

The FFN projects the data into a much higher-dimensional space (usually 4x wider), applies a non-linear activation function (like ReLU), and projects it back down.

$$\text{FFN} (x) = \max (0 , xW_1 + b_1) W_2 + b_2$$

In 2017, the Transformer completely discarded RNNs, effectively saying '***Attention is All You Need***'. Looking at this timeline, do you think we lost anything valuable when we threw away recurrence entirely?

What properties did RNNs have that Transformers have to artificially recreate?

In 2017, the Transformer completely discarded RNNs, effectively saying '***Attention is All You Need***'. Looking at this timeline, do you think we lost anything valuable when we threw away recurrence entirely?

What properties did RNNs have that Transformers have to artificially recreate?

Answer: This gets them thinking about sequence order and recency bias. RNNs naturally understand that word “n” is close to word “n-1” because of time steps. Transformers lose this and have to use complex positional encodings to hack a sense of time back into the system.

Giving Words a Clock

Positional Encoding

Because Transformers process all words simultaneously, they inherently have no concept of word order. "The dog bit the man" looks identical to "The man bit the dog".

To fix this, we inject **Positional Encodings**—unique signals added to the word embeddings to indicate exactly where each word sits in the sequence. We give the model a sense of time.



Encoding Schemes: Static vs. Learned

Feature	Static (Sinusoidal)	Learned (Embeddings)
How it works	Uses overlapping sine and cosine waves of different frequencies.	Uses standard learnable parameters, optimized during training.
Extrapolation	Excellent. Can theoretically process sequence lengths longer than what it was trained on.	Poor. Bounded by the maximum sequence length seen during training.
Adaptability	Fixed mathematical formula; does not adapt to the specific nuances of the dataset.	High. Perfectly molds to the specific structural patterns of the training data.

The Sine Wave Function: (Sinusoidal Positional Encoding)

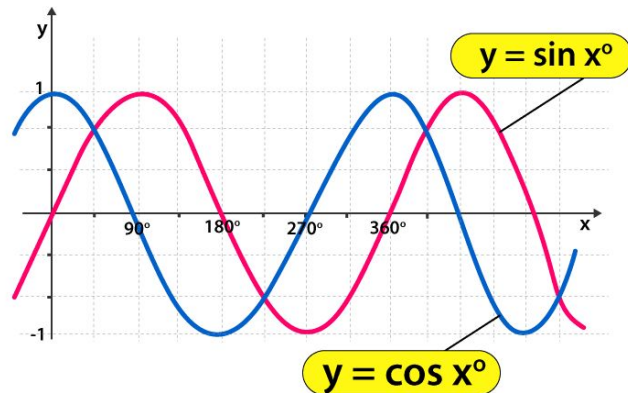
For each position pos and dimension i :

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{\frac{2i}{d_{model}}}}\right)$$

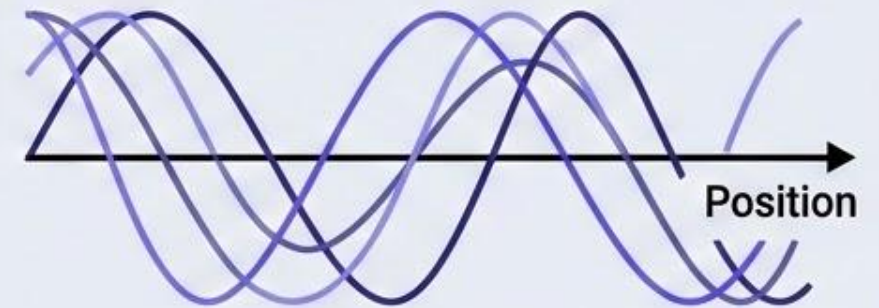
$$PE(pos, 2i + 1) = \cos\left(\frac{pos}{10000^{\frac{2i}{d_{model}}}}\right)$$

Here, d_{model} is the dimension of the model.

- **PE** - the resulting encoding value
- **pos** - the position in the sequence
- **i** - dimension index
- **d_model** - the model's total hidden size



Different Frequencies for Each Dimension



Different Frequencies for Each Dimension

Imagine we train a Transformer using absolute sinusoidal positional encodings with a max context length of 512 tokens. What happens if a user inputs a prompt that is 513 tokens long?

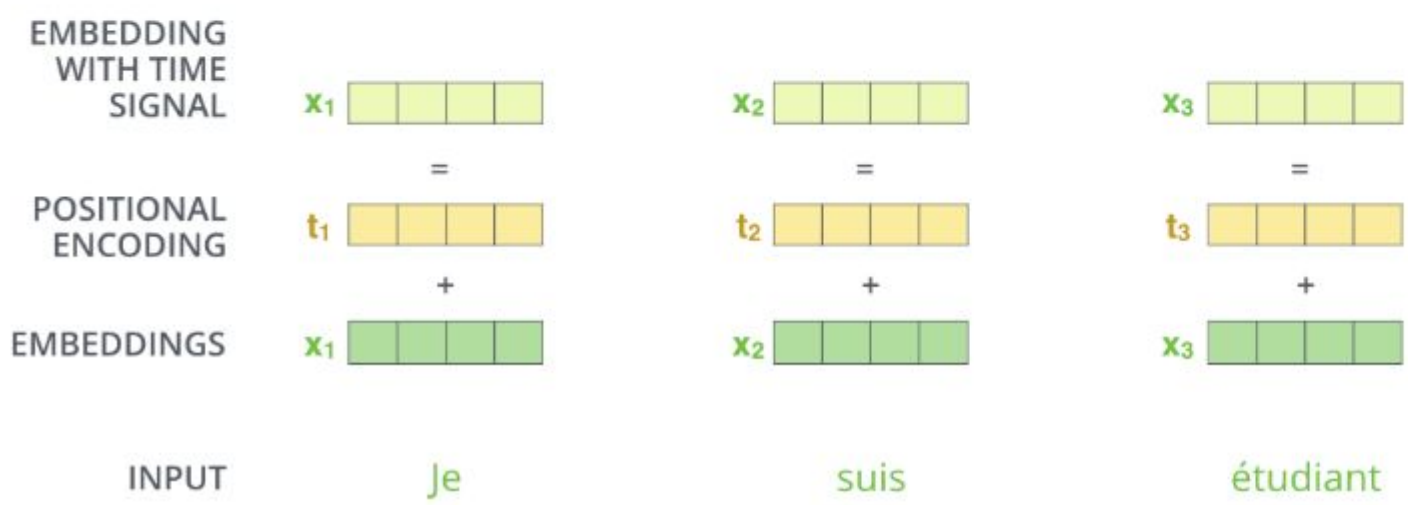
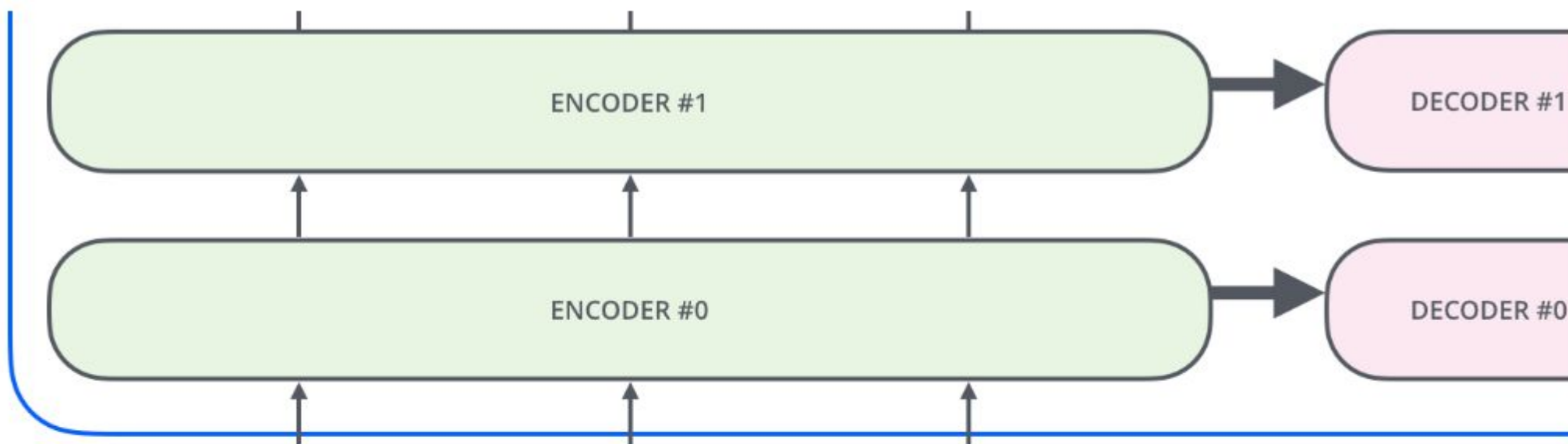
How does the model interpret that 513th token, and how would you fix this limitation?

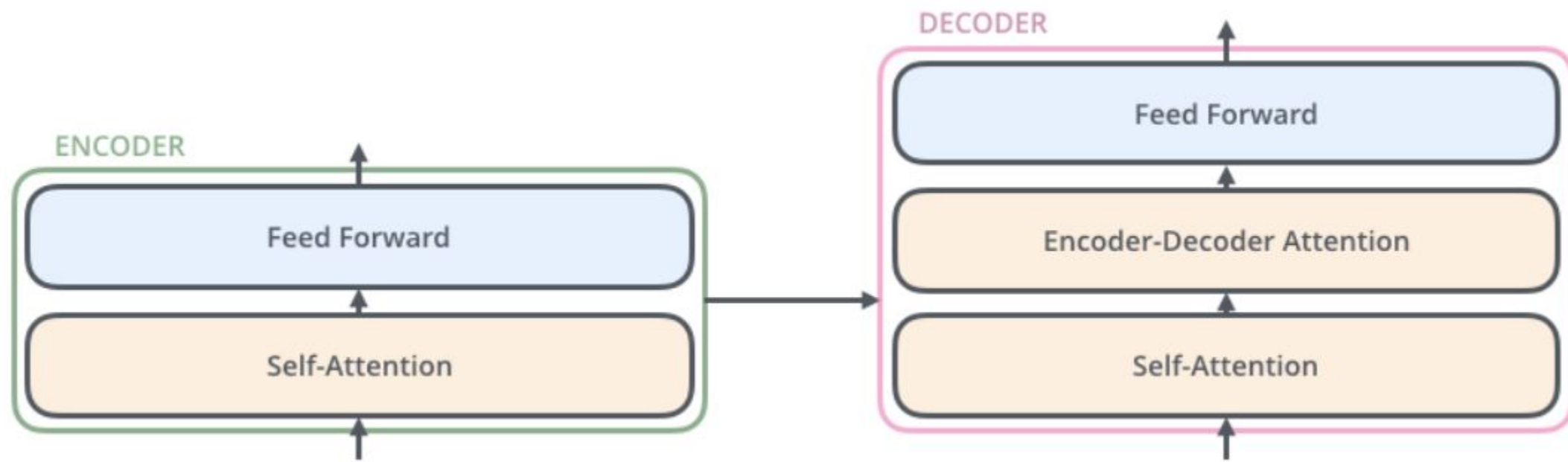
Imagine we train a Transformer using absolute sinusoidal positional encodings with a max context length of 512 tokens. What happens if a user inputs a prompt that is 513 tokens long?

How does the model interpret that 513th token, and how would you fix this limitation?

Answer:

Relative PEs: Ditch absolute coordinates entirely. Upgrade the architecture to RoPE (Rotary Position Embeddings) or ALiBi (Attention with Linear Biases), which inherently compute attention based on the relative distance between words rather than their static page numbers.





Enter the Decoder

What fuels the generation?

While the Encoder processes the entire source sentence at once, the Decoder's job is **auto-regressive generation** (creating output step-by-step). To do this, it requires two specific inputs simultaneously:

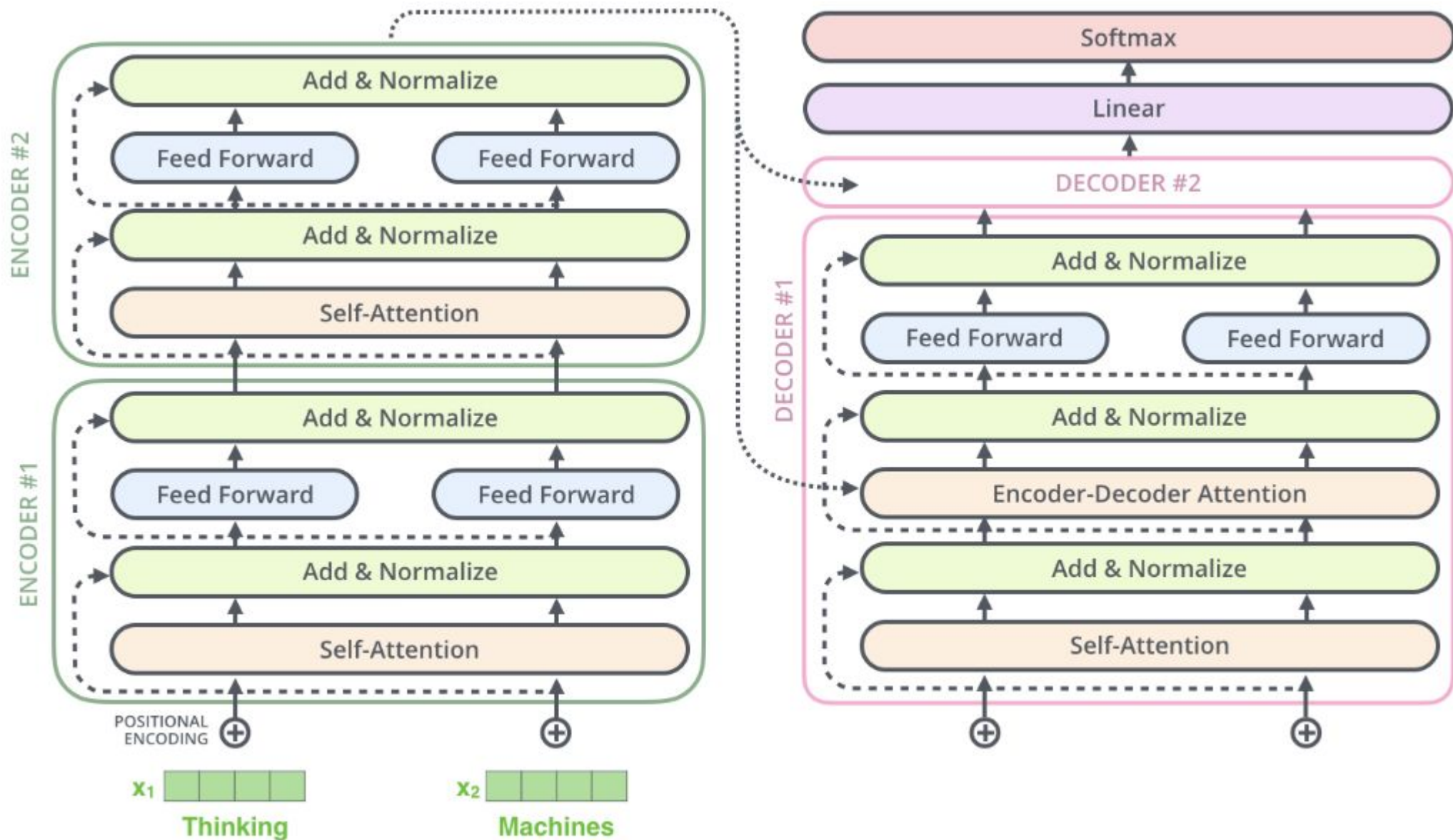
Encoder Output

The continuous, contextualized representation of the full source text. This provides the "memory" of the original message.

Target Sequence

The translation generated *so far* (Shifted Right). For example, to predict the next word, it is fed:

Je suis...



Masked Self-Attention: No Peeking!

Why is it masked?

During training, we feed the entire target sequence at once for speed. However, the model shouldn't "see" future words to predict the current one. It must learn to generate blindly.

How the mask works:

We initialize a mask matrix (a grid of zeros and negative infinities). Before applying the Softmax function to our Attention scores, we add this mask.

$$e^{-\infty} = 0$$

Because the exponent of negative infinity is zero, the Softmax completely neutralizes future tokens, assigning them a 0% probability. The model becomes effectively blind to the future.

Masked attention

		"Dream" x(1)	"big" x(2)	"and" x(3)	"work" x(4)	"for" x(5)	"it" x(6)
"Dream"	x(1)	α_{11}	α_{12}	α_{13}	α_{14}	α_{15}	α_{16}
"big"	x(2)	α_{21}	α_{22}	α_{23}	α_{24}	α_{25}	α_{26}
"and"	x(3)	α_{31}	α_{32}	α_{33}	α_{34}	α_{35}	α_{36}
"work"	x(4)	α_{41}	α_{42}	α_{43}	α_{44}	α_{45}	α_{46}
"for"	x(5)	α_{51}	α_{52}	α_{53}	α_{54}	α_{55}	α_{56}
"it"	x(6)	α_{61}	α_{62}	α_{63}	α_{64}	α_{65}	α_{66}

6x6

For every query, future keys are masked

During inference (when you are chatting with ChatGPT in real-time), do we actually need to apply this negative infinity mask matrix to hide the future tokens? Why or why not?

During inference (when you are chatting with ChatGPT in real-time), do we actually need to apply this negative infinity mask matrix to hide the future tokens? Why or why not?

Answer: No! During inference, the future tokens don't even exist yet, so there is nothing to hide. The mask is a clever trick used strictly during training so we can pass the whole sentence at once without cheating.

The Magic of Cross-Attention

This is where the actual translation occurs. Cross-Attention bridges the source and target languages by mixing inputs from both the Encoder and the Decoder.

The Query (Q)

Comes from the Decoder's previous layer.

Represents the current state of the generated translation.

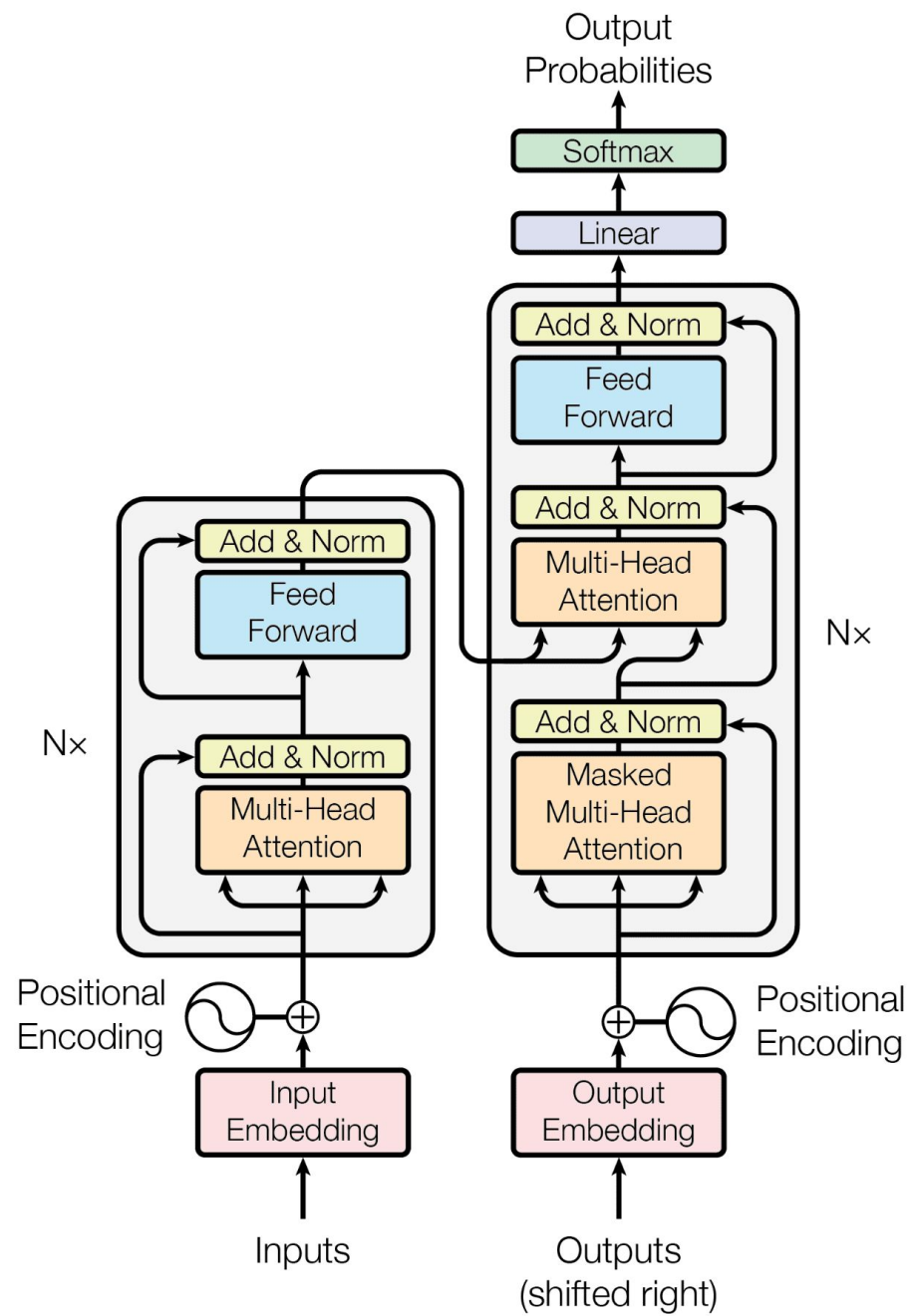
"Based on what I have translated so far, what specific piece of linguistic information do I need from the original source sentence to generate the very next word?"

Keys (K) & Values (V)

Come from the final Encoder layer.

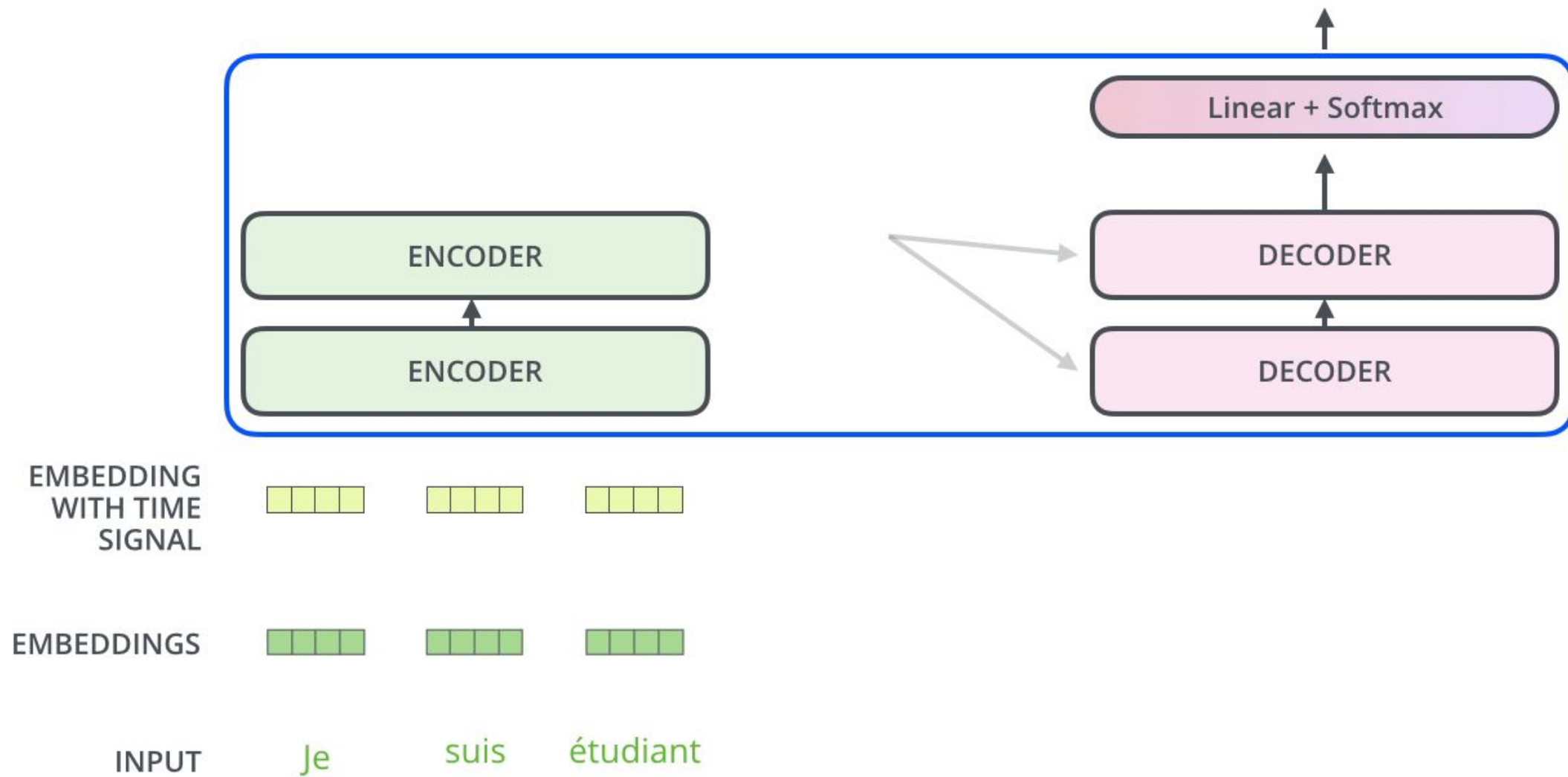
K Acts as an index of the source sentence's grammar and meaning. V hold the actual context.

The Magic: Q pays “attention” to K and V.
Meaning it dynamically retrieves relevant info to generate the next word at each time-step.



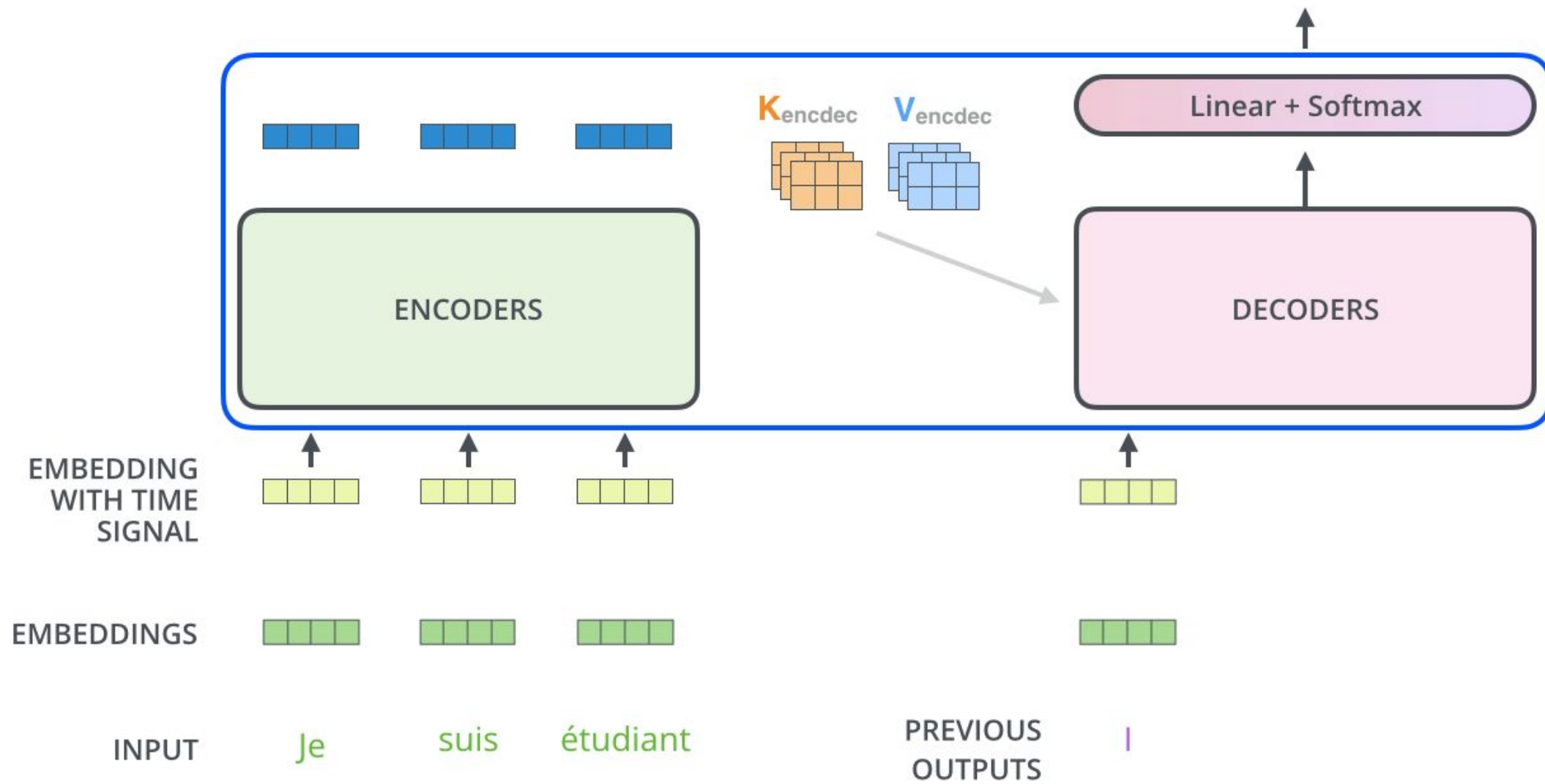
Decoding time step: 1 2 3 4 5 6

OUTPUT



Decoding time step: 1 2 3 4 5 6

OUTPUT |

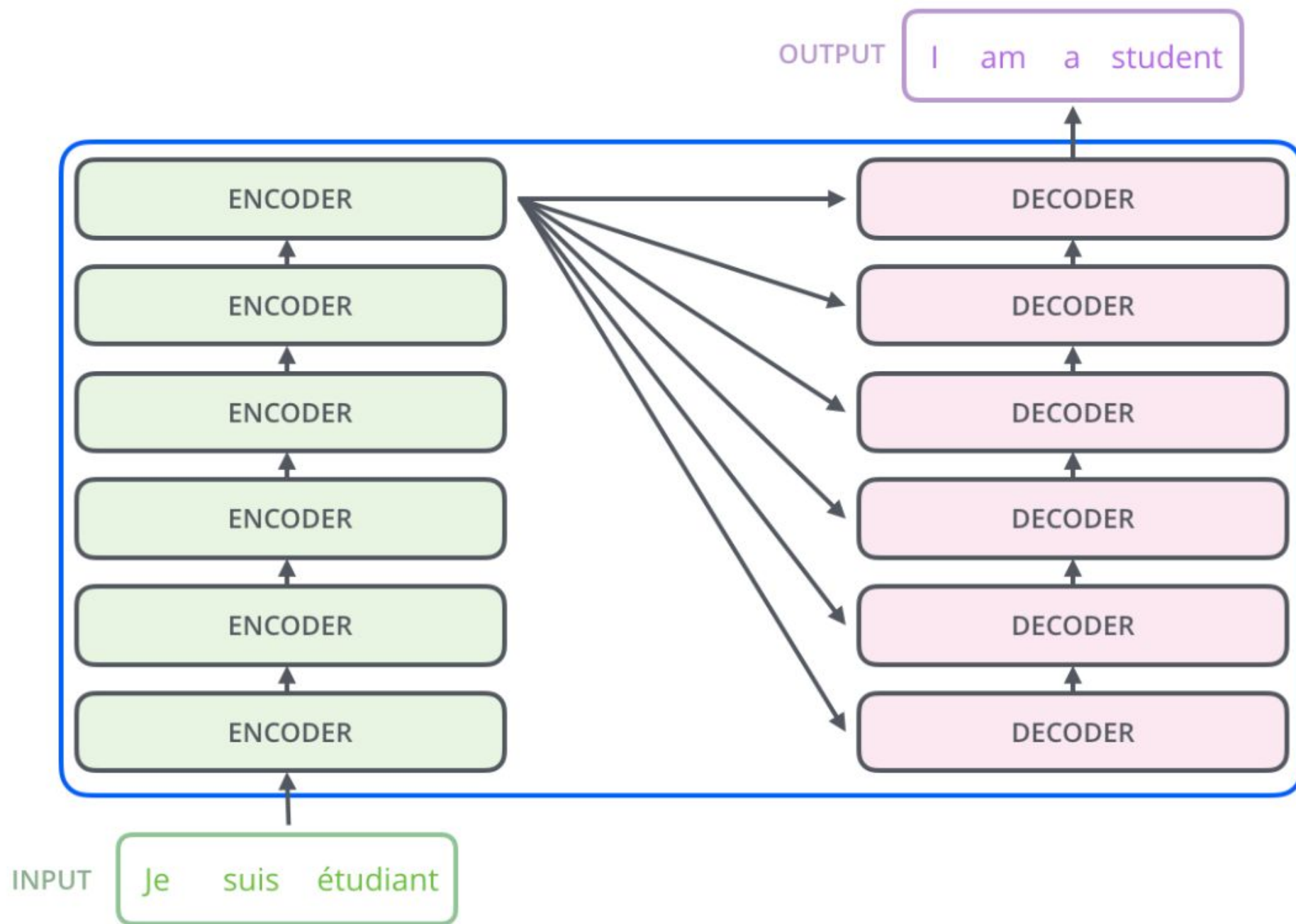


In Cross-Attention, why do the Keys (K) and Values (V) come from the Encoder, while the Queries (Q) come from the Decoder? What if we flipped it?

In Cross-Attention, why do the Keys (K) and Values (V) come from the Encoder, while the Queries (Q) come from the Decoder? What if we flipped it?

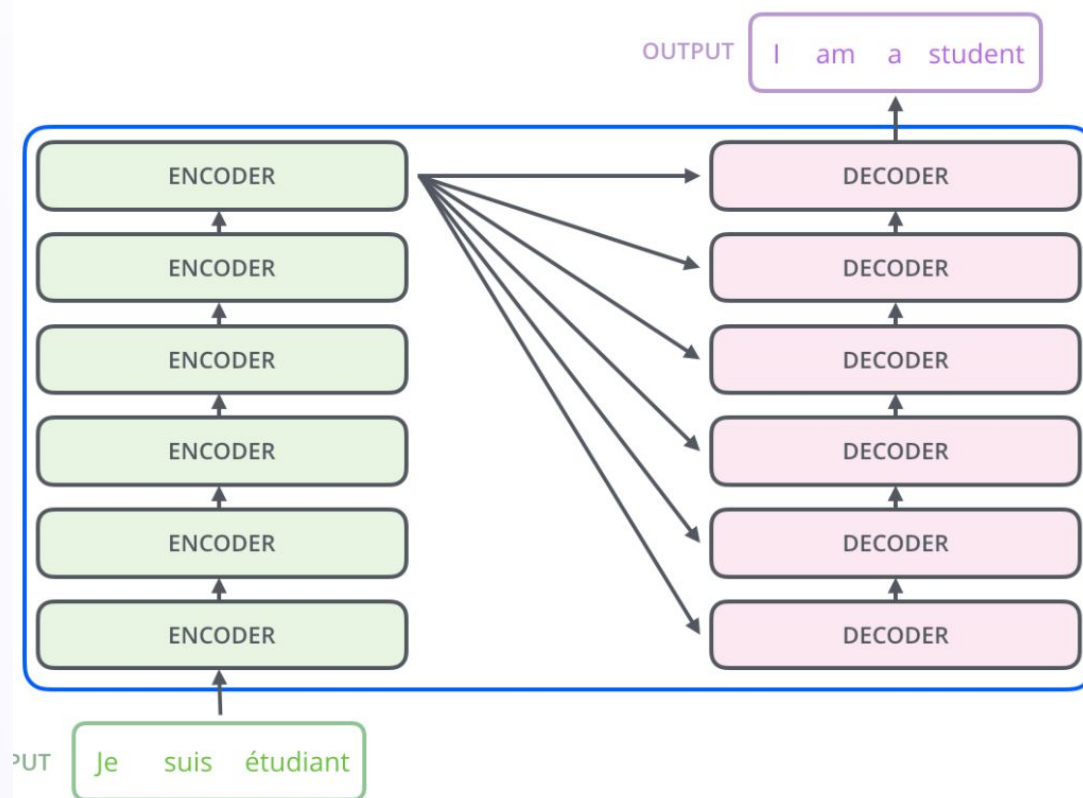
Answer: Flipping it would break the entire directional flow of translation.

The Decoder is the one actively seeking information to generate the next word (hence, issuing the Query), while the Encoder holds the static database of the source language (the Keys and Values).



The Deep Symphony: Stacking Layers

Transformers aren't just a single pass. They utilize deep stacks—commonly 6 to 96 layers of Encoders followed by Decoders.



Encoder Stack

Layer 1 passes to Layer 2, continually refining context. The *absolute final* layer outputs definitive Keys and Values.



Decoder Stack

Every single Decoder layer performs Cross-Attention by querying that *same* final Encoder output.



Refinement

The target representation is enriched layer by layer until it reaches the top of the stack.

The Grand Finale: Softmax Prediction

After traversing all encoder/decoder layers, the final representation passes through a linear projection layer, resulting in "logits"—raw, unnormalized scores for every possible word in the vocabulary.

To convert these arbitrary scores into a clean probability distribution (where all values sum to 100%), we apply the **Softmax** function. The word with the highest probability is our predicted next word.

$$P (w_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

Which word in our vocabulary
is associated with this index?

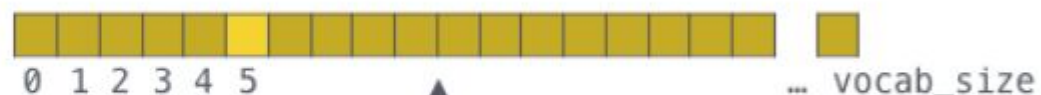
Get the index of the cell
with the highest value
(**argmax**)

Decoder stack output

am

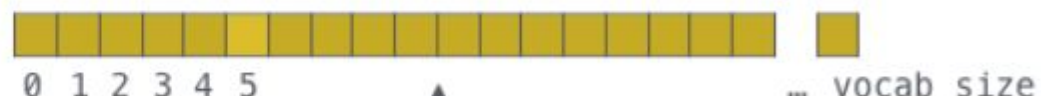
5

log_probs



Softmax

logits

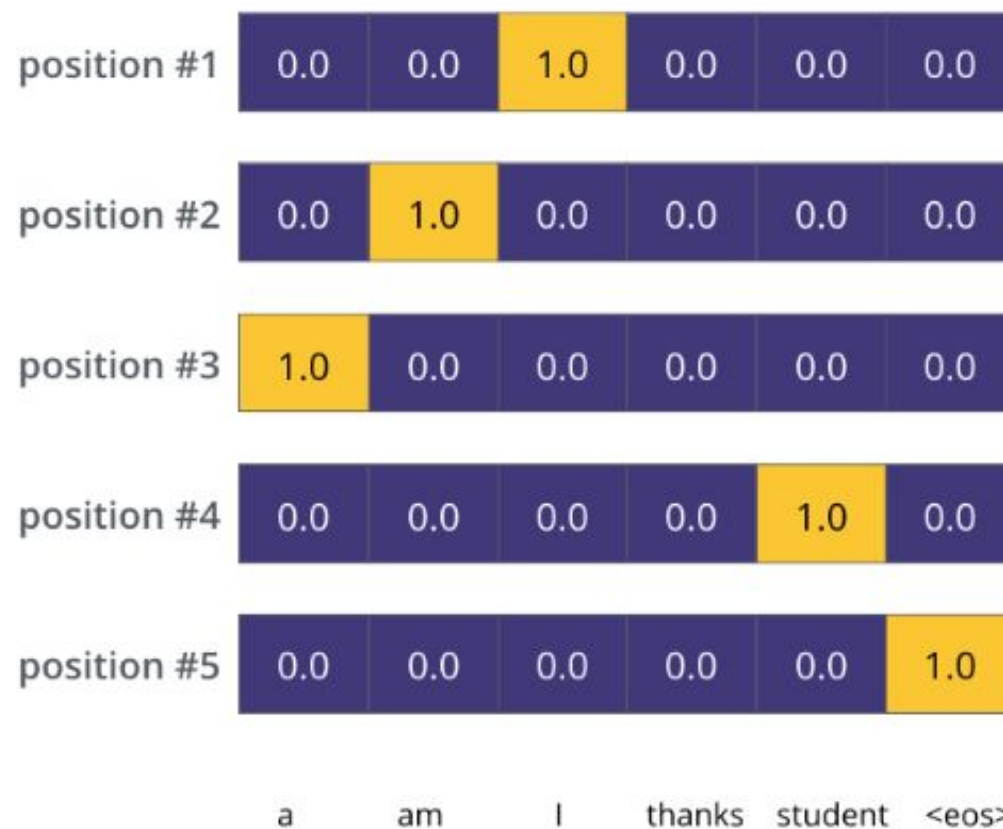


Linear



Target Model Outputs

Output Vocabulary: a am I thanks student <eos>



The targeted probability distributions we'll train our model against in the training example for one sample sentence.

Trained Model Outputs

Output Vocabulary: a am I thanks student <eos>

position #1

0.01	0.02	0.93	0.01	0.03	0.01
------	------	------	------	------	------

position #2

0.01	0.8	0.1	0.05	0.01	0.03
------	-----	-----	------	------	------

position #3

0.99	0.001	0.001	0.001	0.002	0.001
------	-------	-------	-------	-------	-------

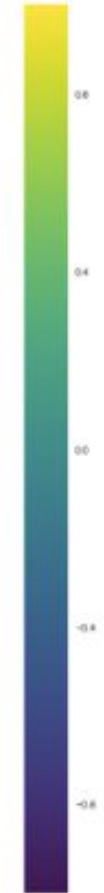
position #4

0.001	0.002	0.001	0.02	0.94	0.01
-------	-------	-------	------	------	------

position #5

0.01	0.01	0.001	0.001	0.001	0.98
------	------	-------	-------	-------	------

a am I thanks student <eos>



Hopefully upon training, the model would output the right translation we expect. Of course it's no real indication if this phrase was part of the training dataset (see: [cross](#)

Teacher Forcing in Sequence Models



THE CHALLENGE: Sequential Errors

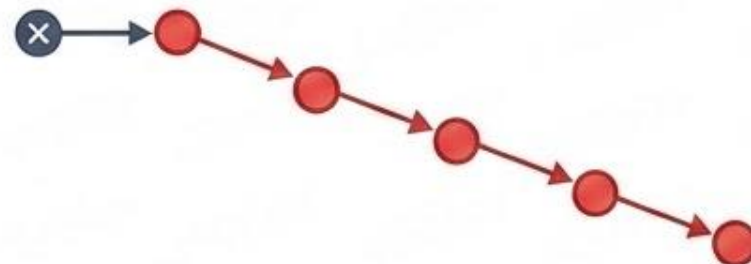
In standard training, a single early mistake propagates, leading the model further astray as it tries to predict subsequent tokens based on its own faulty history.



THE SOLUTION: Ground Truth Guidance

Teacher forcing injects the correct ground truth token from the training data as input for the next step, rather than relying on the model's prediction.

Without Teacher Forcing



With Teacher Forcing



Questions?

Thank you for exploring the complete Transformer architecture.



Kallind Soni

PhD Researcher @IIIT Hyderabad |
Focusing on NLP and Agentic AI | Bu...



Yuvrajsinh Bodana

MS by Research - CSE @ IIIT HYDERABAD

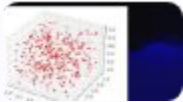


Image Sources



https://static.vecteezy.com/system/resources/thumbnails/066/855/496/small_2x/abstract-network-background-showing-glowing-nodes-connected-by-lines-representing-digital-document-and-data-management-cloud-storage-and-technology-concepts-video.jpg

Source: www.vecteezy.com



<https://static.simonwillison.net/static/2023/embeddings/embeddings.003.jpeg>

Source: simonwillison.net



Thumbnail for
www.vecteezy.com

https://static.vecteezy.com/system/resources/previews/077/190/671/non_2x/abstract-blue-overlapping-sine-waves-on-dark-gradient-background-vector.jpg

Source: www.vecteezy.com



https://png.pngtree.com/background/20231030/original/pngtree-abstract-grid-background-with-interconnected-network-structures-and-big-data-representation-picture-image_5804249.jpg

Source: pngtree.com



https://www.mdpi.com/biomedinformatics/biomedinformatics-05-00037/article_deploy/html/images/biomedinformatics-05-00037-g001-550.jpg

Source: www.mdpi.com



https://elements-resized.envatousercontent.com/elements-video-cover-images/725f11a7-408c-4239-82cd-733d9e8197ad/video_preview/video_preview_0001.jpg?w=500&cf_fit=cover&q=85&format=auto&s=f31c3991d223ed7133cbf4ba28ababd70d0f31c9f3a4987001012f24b5a60a40

Source: elements.envato.com

Image Sources



[https://images.stockcake.com/public/4/2/4/4247d449-861c-4484-98f4-accdd2b6379a_large/ai-layers-visualization-stockcake.](https://images.stockcake.com/public/4/2/4/4247d449-861c-4484-98f4-accdd2b6379a_large/ai-layers-visualization-stockcake)

Source: stockcake.com

<https://bbycroft.net/llm>

https://poloclub.github.io/transformer-explainer/?utm_source=chatgpt.com